

PII Gateway v2 (version2)

Гибридный шлюз маскирования персональных данных (ПД): быстрые детерминированные правила + контекстная оценка уверенности + опциональная NER-модель + span resolver + обратимый токенизированный режим. Заменяет v1 : **итог — та же точность маскирования, но без LLM в пайплайне** и с исправлением scorer-критики («12.03.1998 рядом с ФИО»).

Проектировался под high-load: RPS 1000+ при latency < 100 ms, round-trip `POST` `/process`

без потерь (демаскирование возвращает исходную строку байт-в-байт).

Реализованные фичи

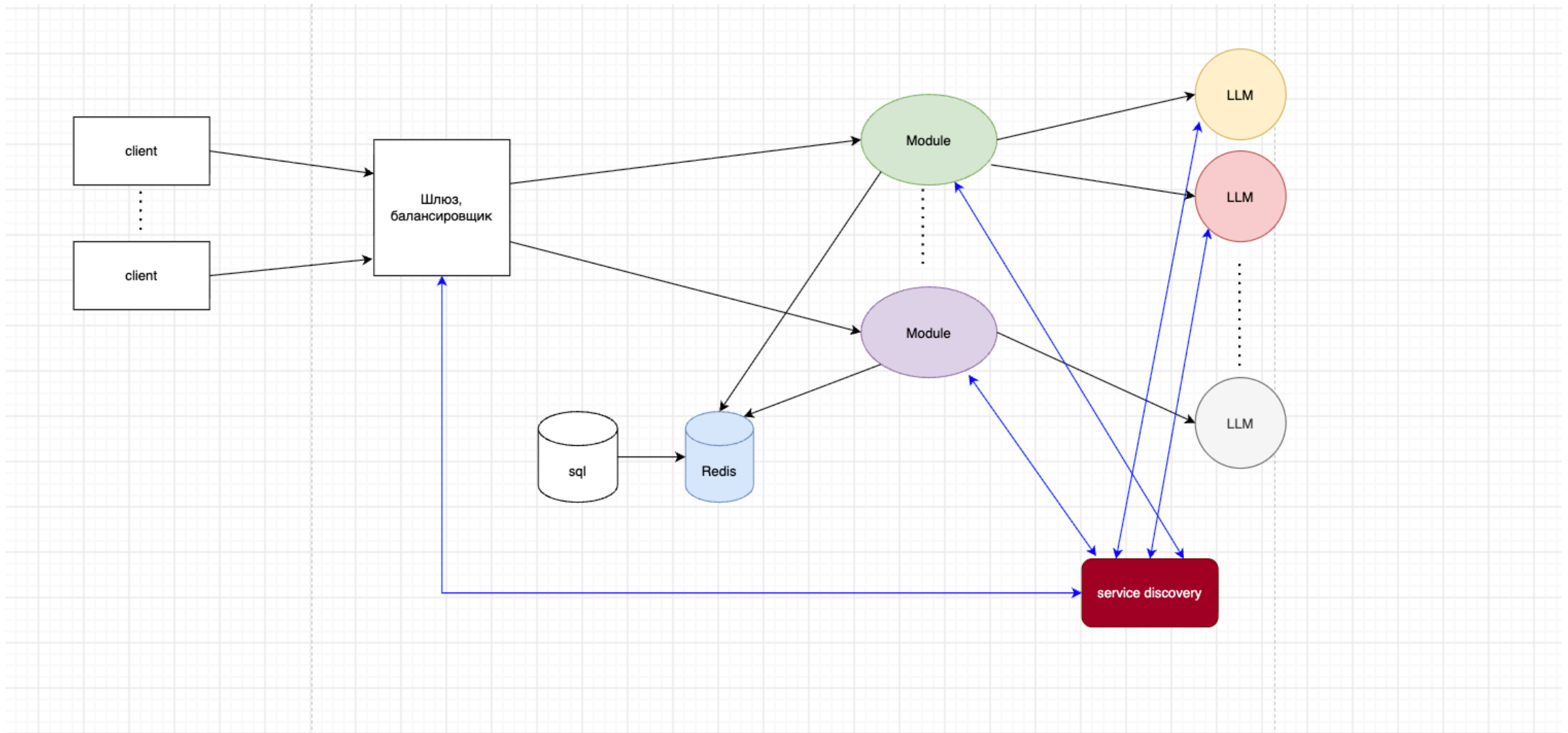
Критерий 3.4 — Гибкая настройка и расширяемость

Фича	Описание
Настройка списка систем-потребителей	Вкл/откл обращения в модуль per-system (<code>enabled</code>), allowlist по имени системы
Настройка перечня типов ПДН по системе	Каждая система маскирует свой набор типов (<code>pii</code>), факт маскирования настраивается
Логирование выявленных типов ПДН	Структурированные логи (slog) по каждому запросу: <code>payload_id</code> , <code>types</code> , <code>latency_ms</code> — без значений ПД
Метрики Latency / RPS / TPS	Prometheus: <code>pii_latency_seconds</code> , <code>pii_requests_total</code> , <code>pii_detected_total</code> +

Ссылки на сервисы

Сервис	URL
Web / Админка (SPA)	http://5.42.118.103:5173/
API (PII Gateway)	http://5.42.118.103:5173/process
Swagger UI	http://5.42.118.103:5173/docs
OpenAPI-спецификация	http://5.42.118.103:5173/openapi.yaml
Health-check	http://5.42.118.103:5173/health
Prometheus-метрики	http://5.42.118.103:5173/metrics
Prometheus UI	http://5.42.118.103:9090
Grafana	http://5.42.118.103:3000 (admin/admin)
Grafana-дашборд «PII	http://5.42.118.103:3000/d/pii-gateway/pii-

Архитектура



Все URL (единый внешний порт :5173)

URL	Назначение
<code>http://5.42.118.103:5173/</code>	Админка (SPA) — список команд, детальные настройки
<code>http://5.42.118.103:5173/process</code>	POST — маскирование/демаскирование
<code>http://5.42.118.103:5173/v1/systems</code>	Управление системами (Bearer-токен)
<code>http://5.42.118.103:5173/v1/config</code>	Read-only конфигурация
<code>http://5.42.118.103:5173/v1/auth/login</code>	Вход в админку
<code>http://5.42.118.103:5173/health</code>	Health-check → <code>ok</code>
<code>http://5.42.118.103:5173/metrics</code>	Prometheus-метрики

Как распознаются персональные данные

Детекция — гибридная: три независимых механизма, объединённых в один `Detector` (`internal/detector`). Ниже — как именно распознаётся каждый тип ПД.

1. Регулярные выражения (fast path, `internal/detector/rules.go`)

Структурированные типы распознаются скомпилированными regex-правилами (`Rule{Re, ContextRe, CaptureRe, Keyword, Priority, Confidence}`). Правила независимы, поэтому для текстов ≥ 4096 байт детекция **параллелится** по ядрам (`detectParallel`), для коротких — последовательно (без оверхеда горутин).

Тип	Паттерн	Примечание
ПАСПОРТ	<code>\d{2}\s?\d{2}\s\d{6}</code>	требуется контекст паспорт/серия
ЗАГРАНПАСПОРТ	<code>\d{2}\s?\d{7}</code>	контекст

Быстрый старт

```
# Локально – без Redis (in-memory store по умолчанию)
```

```
cd version2
```

```
go run ./cmd/server
```

```
# Docker (демон) – memory store, Redis не требуется
```

```
docker compose up -d --build
```

```
# Метрики и дашборды (Prometheus + Grafana)
```

```
docker compose up -d prometheus grafana
```

Проверка:

```
curl -X POST http://5.42.118.103:5173/process \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"payload":"паспорт 4509 123456, email test@example.com","payload_id":"test-1"}'
```

```
# → {"result":"паспорт [ПАСПОРТ], email [EMAIL]}"
```

```
# Демаскирование – тот же payload_id + наша маска возвращает оригинал байт-в-байт
```

```
curl -X POST http://5.42.118.103:5173/process \
```

```
-H "Content-Type: application/json" \
```

Системы-потребители

Поведение модуля настраивается per-system через `config.yaml` — без правки кода.

Запрос указывает систему полем `"system"` ; при указании имени без настроенного API-ключа

система авторизуется неявно.

Авторизация `/process` : если access token (заголовок `X-API-Key` или поле `access_token` в теле) **не указан**, метод остаётся доступным и запрос обрабатывается (200). Указанный токен валидируется: неверный → 401. Только `require_key: true` принудительно требует валидный токен на каждый запрос.

```
systems:
  - name: chat
    api_key: "demo-chat-key"      # непустой => валидируется, если токен указан
    enabled: true                 # false => 403
    pii: [ФИО, ПАСПОРТ, ТЕЛЕФОН] # пусто = все типы
```

Режимы маскирования

Режим	<code>masking.mode</code>	Пример
redact (по умолчанию)	<code>redact</code>	паспорт [ПАСПОРТ], email [EMAIL]
token	<code>token</code>	паспорт [PASSPORT_001], email [EMAIL_002]

В token-режиме `[LABEL_NNN]` — семантически нейтральный обратимый токен (label: `PERSON`, `DATE`, `PHONE`, `EMAIL`, `CARD`, `PASSPORT`, `INN`, `CVV`, `PIN`, `ADDRESS`, `CARDHOLDER`, иначе — имя типа). Нумерация глобальная по отсортированным span-ам, поэтому детерминированная: повторный прогон даёт те же токены. LLM можно передавать текст с токенами, а после ответа восстановить значения через карту

Конфигурация

Файл: `version2/configs/config.yaml`, флаг сервера `-config` (по умолчанию `configs/config.yaml`, относительно рабочей директории).

Полный пример с комментариями

```
# — HTTP —
port: 8080 # HTTP-порт внутреннего сервиса (nginx проксирует :5173)
allow_unmask: true # глобально разрешить демаскирование по существующему payload_id

# — Store (хранилище соответствия token - значение) —
store:
  type: memory # "memory" | "redis"
  ttl_hours: 24 # TTL записей (кроме жизни маски/токена)
  capacity: 262144 # емкость in-memory LRU (FIFO-вытеснение при переполнении)
  redis_url: "redis://localhost:6379/0" # только для store.type: redis

# — Маскирование (глобальный режим по умолчанию) —
masking:
  mode: redact # "redact" | "token" | "synthetic"; per-system может переопределить

# — Контекстный confidence-resolver —
context:
  enabled: true # вкл/выкл контекстную коррекцию уверенности
  boost:
    "ФИО": [клиент, зафичик, паспорт, держатель, анкета, указал] # ключевые слова-сигнаторы (тип: [слова]) - +0.2 за каждое
    "АДРЕС": [адрес клиента, проживает, зарегистрирован] # ключевые слова-сигнаторы
  penalty:
    "АДРЕС": [отделение, офис, банк] # ключевые слова-штрафы - -0.3 за каждое
    "ФИО": [поэт, писатель, написан]

# — Whitelist (известные не-ПД, никогда не маскируются) —
whitelist:
  enabled: true
  persons: [Александр Пушкин, Лев Толстой, Антон Чехов]
  addresses: [Красная площадь, отделение Альфа-Банка]
  organizations: [Альфа-Банк]

# — Resolve (приоритеты типов при пересечении span-ов) —
resolve:
  priority: # выше = побеждает при равной длине
    "КАРТА": 100
    "ПАСПОРТ": 95
    "ИНН": 40

# — NER (опциональная ML-модель, smart path) —
ml:
  enabled: false # NER: https://github.com/yalue/onnruntime-go
  model_path: "" # rubert-tiny BERT-модель + vocab + labels
  vocab_path: ""
  labels: [O, B-PER, I-PER, B-LOC, I-LOC, B-ORG, I-ORG]

# — Sensitive-типы (одиночный тип маскируется только при другой ПД рядом) —
sensitive:
  - ПИН
  - CVV

# — Rate limit (0 = выключено) —
rate_limit:
  rps: 0 # 0 - выключено; иначе 429 при превышении
  burst: 0

# — Системы-потребители —
# system="" (или не известное имя) -> поведение по умолчанию (глобальные настройки).
systems:
  - name: chat
    api_key: "demo-chat-key" # не пустой => валидируется, если token указан
    enabled: true # false => 403
    pii: [ФИО, ПАСПОРТ, ТЕЛЕФОН, EMAIL, КАРТА, ИНН, ДАТА] # пусто = все типы
    masking: token # "redact" | "token" | "synthetic"; пусто = глобальный
    allow_unmask: true # разрешено ли демаскирование
    require_key: false # true => token обязателен (иначе 401)
  - name: analytics
    api_key: "demo-analytics-key"
    enabled: true
    pii: [ТЕЛЕФОН, EMAIL] # аналитике недоступны ФИО/паспорт/карты
    allow_unmask: false # аналитике запрещено демаскирование
    masking: redact
  - name: external_llm
    api_key: "demo-llm-key"
    enabled: true
    pii: [] # пусто = маскировать все типы
```

Админка и управление системами (PostgreSQL)

Системы-потребители, overlay-правила и комбинации хранятся в PostgreSQL (`config.Database.DSN`). При первом старте пустые таблицы заполняются из `config.yaml` (seed); дальше БД — источник истины. Админка защищена логином и паролем (`config.Admin.Login` / `Password` , по умолчанию `admin` / `admin123` , bcrypt-хэш в таблице `admins`).

Команды

analytics

включена · режим: redact · типов ПД: 2

chat

включена · режим: token · типов ПД: 6

NER (smart path, опционально)

`ml.enabled: true` + пути к ONNX-модели и vocab. NER подключается к детектору и используется для неоднозначных случаев; при недоступности модели сервер стартует

без неё (`slog.Warn` , graceful degradation). Всё остальное — rule-based и fast path.

Для установки модели см. `scripts/setup_ner.sh` из v1.

Производительность

```
cd version2
go run ./cmd/bench -c 100 -d 10s          # короткий payload (по умолчанию)
go run ./cmd/bench -c 20 -d 6s -size 100000 # длинный текст до 100k символов
```

Bench отправляет реалистичный payload паспорт 4509 123456, email ..., телефон +7 ...

с уникальным payload_id на каждый запрос (всегда путь маскирования). Флаг -size N

повторяет ПД-фрагмент до ~N символов (UTF-8), покрывая порог 4096 байт, после которого детектор переключается на параллельную по правилам обработку. Клиент переиспользует keep-alive-соединения (читает тело ответа) и пул idle-коннектов масштабируется вместе с -c. Примеры на данной машине:

```
$ go run ./cmd/bench -c 100 -d 10s
total=402584 ok=402584 fail=0 rps=40242
```

Тестирование

```
go build ./...      # сборка
go vet ./...        # статический анализ
go test ./...       # unit + integration + accuracy
```